From Excel to MySQL to Tableau

Notebook walkthrough · pandas · mysql-connector-python · Tableau Desktop

This guide loads the **Orders** sheet of the Superstore Excel workbook into a MySQL database called `tableau_data`, then connects Tableau to it. The pattern is common in real BI work: data arrives as a spreadsheet, Python checks and cleans it, a database stores it as the single trusted copy, and Tableau reads from the database.

Each code block is one notebook cell. Every cell was run against a MySQL-compatible server and the output shown is real.



Step	What you do	Result
0	Install libraries; keep credentials in a `.env` file	a ready notebook
1	Read the `Orders` sheet	a DataFrame of 9,994 rows × 21 columns
2	Clean and check the data	trimmed text, 5-digit postal codes, real dates
3	Connect and create the database `tableau_data`	an empty database
4	Create the table `orders` with the Excel column names	an empty table with the right types
5	Insert the cleaned rows and verify	9,994 rows; totals match Excel
6	Connect Tableau Desktop to `tableau_data.orders`	a live data source and a first chart

Part 0 Setup

VS CODE TERMINAL (OR A NOTEBOOK CELL WITH %pip)

```
pip install pandas xlrd mysql-connector-python python-dotenv
```

`xlrd` reads the older `.xls` format (for `.xlsx` files pandas uses `openpyxl`). `python-dotenv` reads your credentials from a `.env` file in the project folder:

`.env`

```
MYSQL_HOST=localhost
MYSQL_USER=your_mysql_username
MYSQL_PASSWORD=your_mysql_password
```

> **Watch out:** Add `.env` to your `.gitignore` so the password never reaches GitHub. Put the notebook, the `.env` file and `Sample - Superstore.xls` in the same folder.

Part 1 Read the Orders sheet

c1_read.py

```
import pandas as pd

excel = pd.ExcelFile("Sample - Superstore.xls")
print(excel.sheet_names)

orders = pd.read_excel(excel, sheet_name="Orders")
print(orders.shape)
```

OUTPUT

```
['Orders', 'Returns', 'People']
(9994, 21)
```

The workbook has three sheets; `sheet_name="Orders"` picks the one we need. 9,994 order lines and 21 columns.

c1_dtypes.py

```
print(orders.dtypes)
```

OUTPUT

```
Row ID                  int64
Order ID                  str
Order Date       datetime64[us]
Ship Date        datetime64[us]
Ship Mode                 str
Customer ID               str
Customer Name             str
Segment                   str
Country                   str
City                      str
State                     str
Postal Code             int64
Region                    str
Product ID                str
Category                  str
Sub-Category              str
Product Name              str
Sales                 float64
Quantity                int64
Discount              float64
Profit                float64
dtype: object
```

c1_head.py

```
print(orders.iloc[:3, :8])
```

OUTPUT

```
   Row ID        Order ID Order Date  ... Customer ID    Customer Name    Segment
0       1  CA-2016-152156 2016-11-08  ...    CG-12520      Claire Gute   Consumer
1       2  CA-2016-152156 2016-11-08  ...    CG-12520      Claire Gute   Consumer
2       3  CA-2016-138688 2016-06-12  ...    DV-13045  Darrin Van Huff  Corporate

[3 rows x 8 columns]
```

Part 2 Clean and check the data

Before anything goes into a database, check it. The database will become the trusted copy that every Tableau dashboard reads.

Missing values and duplicates

c2_missing.py

```
print("Missing values:", orders.isna().sum().sum())
print("Duplicate rows:", orders.duplicated().sum())
print("Duplicate Row IDs:", orders["Row ID"].duplicated().sum())
```

OUTPUT

```
Missing values: 0
Duplicate rows: 0
Duplicate Row IDs: 0
```

No gaps and no repeats. `Row ID` is unique, so it can be the table's primary key.

Stray spaces in text

c2_strip.py

```
text_cols = orders.select_dtypes(include="str").columns
untrimmed = {col: int((orders[col] != orders[col].str.strip()).sum()) for col in text_cols}
print({col: n for col, n in untrimmed.items() if n > 0})

orders[text_cols] = orders[text_cols].apply(lambda col: col.str.strip())
print("After strip:", int((orders["Product Name"] != orders["Product
Name"].str.strip()).sum()))
```

OUTPUT

```
{'Product Name': 16}
After strip: 0
```

A trailing space makes "Staples " and "Staples" two different products in Tableau. On older pandas use `include="object"`.

Postal codes lost their leading zero

c2_postal.py

```
print(orders.loc[orders["State"] == "Vermont", ["City", "State", "Postal Code"]].head(2))
print("Codes with fewer than 5 digits:", (orders["Postal Code"] < 10000).sum())

orders["Postal Code"] = orders["Postal Code"].astype(str).str.zfill(5)
print(orders.loc[orders["State"] == "Vermont", "Postal Code"].head(2).tolist())
```

OUTPUT

```
            City    State  Postal Code
2234  Burlington  Vermont         5408
5274  Burlington  Vermont         5408
Codes with fewer than 5 digits: 449
['05408', '05408']
```

Excel stored postal codes as numbers, so Burlington's `05408` became `5408`. A postal code is a **label, not a quantity**: store it as text and pad it back to five digits with `zfill(5)`. Tableau can then map it correctly.

Dates and value checks

c2_checks.py

```
orders["Order Date"] = pd.to_datetime(orders["Order Date"]).dt.date
orders["Ship Date"] = pd.to_datetime(orders["Ship Date"]).dt.date

print("Order dates:", orders["Order Date"].min(), "to", orders["Order Date"].max())
print("Shipped before ordered:", (orders["Ship Date"] < orders["Order Date"]).sum())
print("Quantity <= 0:", (orders["Quantity"] <= 0).sum())
print("Discount outside 0-1:", (~orders["Discount"].between(0, 1)).sum())
print("Rows with a loss:", (orders["Profit"] < 0).sum())
```

OUTPUT

```
Order dates: 2014-01-03 to 2017-12-30
Shipped before ordered: 0
Quantity <= 0: 0
Discount outside 0-1: 0
Rows with a loss: 1871
```

`.dt.date` keeps only the date part, matching a MySQL `DATE` column. Negative profit is a real business result (discounted sales), not an error, so those rows stay.

Record the totals for later

c2_totals.py

```
print("Rows:", len(orders))
print("Total sales:", round(orders["Sales"].sum(), 2))
print("Total profit:", round(orders["Profit"].sum(), 2))
```

OUTPUT

```
Rows: 9994
Total sales: 2297200.86
Total profit: 286397.02
```

After loading into MySQL, and again in Tableau, these three numbers must match.

Part 3 Connect and create the database tableau_data

Your connection cell, with the `database` line left out because the database does not exist yet:

c3_database.py

```
import os
import mysql.connector
from dotenv import load_dotenv, find_dotenv
load_dotenv(find_dotenv())

conn = mysql.connector.connect(
    host=os.environ.get("MYSQL_HOST"),
    user=os.environ.get("MYSQL_USER"),
    password=os.environ.get("MYSQL_PASSWORD"),
)
print("Connected:", conn.is_connected())

cursor = conn.cursor()
cursor.execute("""
    CREATE DATABASE IF NOT EXISTS tableau_data
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
""")
cursor.execute("SHOW DATABASES LIKE 'tableau%'")
print(cursor.fetchall())

cursor.execute("USE tableau_data")
```

OUTPUT

```
Connected: True
[('tableau_data',)]
```

`utf8mb4` stores every character correctly. 278 product names contain characters such as `é` or `™`, which older character sets damage. `USE tableau_data` makes it the current database for the following cells, the same as reconnecting with `database="tableau_data"`.

Part 4 Create the table orders with the Excel column names

MySQL allows spaces and hyphens in column names, such as **Order ID** and **Sub-Category**, if each name is wrapped in **backticks** (the key to the left of 1), as in the cell below. Keeping the Excel names means Tableau shows the same field names your learners already know. Remember the backticks in every later query that uses these names.

c4_create.py

```
cursor.execute("DROP TABLE IF EXISTS orders")
cursor.execute("""
    CREATE TABLE orders (
        `Row ID`        INT            NOT NULL PRIMARY KEY,
        `Order ID`      VARCHAR(20)    NOT NULL,
        `Order Date`    DATE           NOT NULL,
        `Ship Date`     DATE           NOT NULL,
        `Ship Mode`     VARCHAR(20)    NOT NULL,
        `Customer ID`   VARCHAR(10)    NOT NULL,
        `Customer Name` VARCHAR(50)    NOT NULL,
        `Segment`       VARCHAR(20)    NOT NULL,
        `Country`       VARCHAR(30)    NOT NULL,
        `City`          VARCHAR(40)    NOT NULL,
        `State`         VARCHAR(30)    NOT NULL,
        `Postal Code`   VARCHAR(10)    NOT NULL,
        `Region`        VARCHAR(10)    NOT NULL,
        `Product ID`    VARCHAR(20)    NOT NULL,
        `Category`      VARCHAR(20)    NOT NULL,
        `Sub-Category`  VARCHAR(20)    NOT NULL,
        `Product Name`  VARCHAR(255)   NOT NULL,
        `Sales`         DECIMAL(12, 4) NOT NULL,
        `Quantity`      INT            NOT NULL,
        `Discount`      DECIMAL(4, 2)  NOT NULL,
        `Profit`        DECIMAL(12, 4) NOT NULL
    )
""")
cursor.execute("DESCRIBE orders")
for field, col_type, null, key, default, extra in cursor.fetchall():
    print(f"{field:<14} {col_type:<15} {key}")
```

OUTPUT

```
Row ID         int(11)         PRI
Order ID       varchar(20)
Order Date     date
Ship Date      date
Ship Mode      varchar(20)
Customer ID    varchar(10)
Customer Name  varchar(50)
Segment        varchar(20)
Country        varchar(30)
City           varchar(40)
State          varchar(30)
Postal Code    varchar(10)
Region         varchar(10)
Product ID     varchar(20)
Category       varchar(20)
Sub-Category   varchar(20)
Product Name   varchar(255)
Sales          decimal(12,4)
Quantity       int(11)
Discount       decimal(4,2)
Profit         decimal(12,4)
```

Choice	Why
`DROP TABLE IF EXISTS` first	the cell can be rerun from scratch without errors (it deletes the old table, so never run it on live data)
`Row ID` as `PRIMARY KEY`	it is unique; MySQL rejects an accidental duplicate load
`Postal Code` as `VARCHAR`	keeps leading zeros
`DATE` for the two dates	Tableau recognises them as dates for timelines and filters
`DECIMAL` for money and rates	exact values; no floating-point rounding in totals
`VARCHAR` sizes	from the longest value in each column (`orders[col].str.len().max()`), with room to spare

Part 5 Insert the cleaned data

c5_prepare.py

```
columns = ", ".join(f"`{col}`" for col in orders.columns)
placeholders = ", ".join(["%s"] * len(orders.columns))
insert_sql = f"INSERT INTO orders ({columns}) VALUES ({placeholders})"
print(insert_sql[:120], "...")

rows = orders.astype(object).values.tolist()
print(rows[0])
```

OUTPUT

```
INSERT INTO orders (`Row ID`, `Order ID`, `Order Date`, `Ship Date`, `Ship Mode`, `Customer
ID`, `Customer Name`, `Segme ...
[1, 'CA-2016-152156', datetime.date(2016, 11, 8), datetime.date(2016, 11, 11), 'Second
Class', 'CG-12520', 'Claire Gute', 'Consumer', 'United States', 'Henderson', 'Kentucky',
'42420', 'South', 'FUR-BO-10001798', 'Furniture', 'Bookcases', 'Bush Somerset Collection
Bookcase', 261.96, 2, 0.0, 41.9136]
```

The `INSERT` is built from the DataFrame's own column names, so it always matches the table. `astype(object).values.tolist()` turns each row into plain Python values, which the connector can send (it cannot send some NumPy number types).

c5_insert.py

```
batch_size = 1000
for start in range(0, len(rows), batch_size):
    cursor.executemany(insert_sql, rows[start:start + batch_size])
conn.commit()

cursor.execute("SELECT COUNT(*) FROM orders")
print("Rows in tableau_data.orders:", cursor.fetchone()[0])
```

OUTPUT

```
Rows in tableau_data.orders: 9994
```

Sending 1,000 rows at a time keeps each request a manageable size. Nothing is saved until `conn.commit()`.

Verify the load

c5_verify.py

```
cursor.execute("SELECT COUNT(*), ROUND(SUM(Sales), 2), ROUND(SUM(Profit), 2) FROM orders")
print(cursor.fetchone())

cursor.execute("SELECT `City`, `Postal Code` FROM orders WHERE `State` = 'Vermont' LIMIT
2")
print(cursor.fetchall())

cursor.execute("""
    SELECT `Region`, COUNT(*) AS order_lines, ROUND(SUM(`Sales`), 2) AS sales
    FROM orders GROUP BY `Region` ORDER BY sales DESC
""")
for row in cursor.fetchall():
    print(row)

cursor.close()
conn.close()
```

OUTPUT

```
(9994, Decimal('2297200.86'), Decimal('286397.02'))
[('Burlington', '05408'), ('Burlington', '05408')]
('West', 3203, Decimal('725457.82'))
('East', 2848, Decimal('678781.24'))
('Central', 2323, Decimal('501239.89'))
('South', 1620, Decimal('391721.91'))
```

Same row count, same sales and profit totals as in pandas, and the postal codes keep their leading zeros. The database is ready.

> **Shortcut:** With SQLAlchemy installed, pandas can do steps 4 and 5 in one line: `orders.to_sql("orders", engine, if_exists="replace", index=False)`. It is quicker, but pandas then chooses the column types itself, which is why this guide creates the table explicitly.

Part 6 Connect Tableau to tableau_data.orders

> **Watch out: Use Tableau Desktop**, not Tableau Public. Tableau Public cannot connect directly to a MySQL database. Students can request a free Tableau Desktop licence through Tableau for Students, or use the free trial.

One-time: install the MySQL driver

- Tableau needs the **MySQL ODBC driver (MySQL Connector/ODBC)** to talk to MySQL.
- In Tableau, choose **Connect › To a Server › MySQL**. If the driver is missing, the dialog shows a link to Tableau's driver download page with the right version and installation steps for your system.
- On Windows install the 64-bit version; restart Tableau after installing.

Connect

1. Open Tableau Desktop. In the **Connect** pane, under **To a Server**, click **More...** if needed, then **MySQL**.

2. Fill in the connection dialog (table below) and click **Sign In**.
3. On the data source page, choose **tableau_data** from the **Database** drop-down.
4. Under **Table**, drag **orders** onto the canvas. The preview grid shows the rows with the Excel column names.
5. Choose **Live** (always reads the current database) or **Extract** (a fast snapshot that you refresh). For class, Live shows changes made from Python straight away.
6. Click the **Sheet 1** tab to start building.

Field	Enter	Note
Server	`localhost` (or `127.0.0.1`)	the machine running MySQL; same as `MYSQL_HOST` in `.env`
Port	`3306`	MySQL's default
Username	your MySQL user	same as `MYSQL_USER`
Password	your MySQL password	same as `MYSQL_PASSWORD`
Require SSL	leave unticked	for a local server; a company server may require it

First check: do the numbers match?

1. Drag **Region** to Rows and **Sales** to Columns. Tableau sums Sales by default.
2. Compare with Step 5: West 725,457.82, East 678,781.24, Central 501,239.89, South 391,721.91.
3. Total sales should be **2,297,200.86**. If not, check that you loaded the cleaned data once (9,994 rows) and that Sales is a measure (green), not a dimension.

Tableau shows `Order Date` and `Ship Date` with a calendar icon because they are `DATE` columns, and `Postal Code` as text, which it can assign the **ZIP Code/Postcode** geographic role for maps.

Troubleshooting

Symptom	Likely cause	Fix
Dialog asks for a driver	MySQL ODBC driver not installed	install it from the link in the dialog; restart Tableau
Cannot connect to server	MySQL not running, or wrong server/port	start MySQL (Workbench can connect?); check `localhost` and `3306`
Access denied	wrong user or password	use the same values as the `.env` file
Authentication plugin error	an older driver with MySQL 8's default login method	install a current MySQL Connector/ODBC version
`tableau_data` not in the list	notebook cells not run, or a different MySQL server	rerun Steps 3–5; confirm the same server in Workbench
MySQL missing from the Connect pane	using Tableau Public	use Tableau Desktop
Mac cannot find the server	relative server name	use `localhost` or a fully qualified server name

Summary

Step	Key code or action
Read	`pd.read_excel("Sample - Superstore.xls", sheet_name="Orders")`
Clean	`str.strip()`, `astype(str).str.zfill(5)`, `pd.to_datetime(...).dt.date`, checks
Database	`CREATE DATABASE IF NOT EXISTS tableau_data CHARACTER SET utf8mb4`
Table	`CREATE TABLE orders (...)` with each Excel column name in backticks; `Row ID` is the primary key
Load	`cursor.executemany(insert_sql, rows)` in batches, then `conn.commit()`
Verify	row count and `SUM(Sales)` match pandas: 9,994 rows, 2,297,200.86
Tableau	Connect › MySQL › localhost:3306 › Database `tableau_data` › drag `orders`